TAMBERBOX

Complete Build Journal & Technical Guide

Orange Pi 5 Ultra · Armbian Noble · Hyprland · Local AI

Built by Praknoor(Aka Gurmoor Tamber) · Every error documented · Every fix included

What Tamberbox Is

Tamberbox is a fully custom AI-powered laptop built on the Orange Pi 5 Ultra single-board computer. Every piece of software was hand-installed, compiled, and configured — nothing was pre-built. This document records the complete journey including every error encountered and exactly how it was fixed.

✅ ACHIEVED IN THIS BUILD

- Armbian Noble (Ubuntu 24.04) — Vendor kernel 6.1.115 with full RK3588 hardware support
- Hyprland 0.47.0 — Compiled from source on ARM64 (no pre-built binary exists for Ubuntu Noble ARM64)
- Dog Samurai SDDM login screen with animated video background — fixed Qt5 mediaservice missing package
- Plymouth Tamberbox boot splash — Cross HUD animation with TAMBERBOX branding
- Ollama local AI — qwen2.5:1.5b model with custom Tamberbox personality built by Praknoor
- Whisper.cpp — Offline speech-to-text compiled from source with ARM NEON optimizations
- RKNN Toolkit2 — 6 TOPS NPU enabled and verified working
- Tamberbox AI GUI — Custom web interface with voice input, system stats, speak mode
- Sci-fi MOTD terminal banner with live system info
- Pywal dynamic color theming from wallpaper
- mpvpaper live animated wallpaper (Zenitsu)
- Cinnamon cursor theme
- PS4 UI gaming mode (in progress)

Hardware spec

- SBC: Orange Pi 5 Ultra
- CPU: Rockchip RK3588 — 4x Cortex-A76 @ 2.4GHz + 4x Cortex-A55 @ 1.8GHz
- RAM: 8GB LPDDR5
- GPU: Mali-G610 MP4 — Panthor driver (open source)
- NPU: 6 TOPS — RKNN Toolkit2 — INT4/INT8/INT16/FP16
- Storage: microSD (boot) + M.2 NVMe PCIe 3.0 x4
- Network: Wi-Fi 6E + Bluetooth 5.3
- OS: Armbian 26.2.1 Noble (Ubuntu 24.04 LTS)
- Kernel: 6.1.115-vendor-rk35xx (Rockchip BSP)

Phase 1 — Flash & First Boot

Image used

Armbian_26.2.1_Orangepi5_noble_vendor_6.1.115_gnome_desktop.img.xz

> ⚠ *This image targets 'Orangepi5' not 'Orangepi5ultra' — use it anyway, the DTB file correctly identifies the Ultra: fdtfile=rockchip/rk3588-orangepi-5-ultra.dtb*
>
> ⚠ *Image came with GNOME desktop — we strip it completely and replace with Hyprland*

STEP1 Flash with balenaEtcher

Windows — balenaEtcher handles .xz decompression automatically

> ✓ *balenaEtcher decompresses .xz automatically. Do NOT extract first.*

STEP2 First boot setup

Armbian wizard — set root password and create user

```
# Default credentials if wizard doesn't run:
login: root
password: 1234

# Create user
useradd -m -G wheel,audio,video,storage,network -s /bin/bash praknoor
passwd praknoor
```

STEP3 Connect Wi-Fi

```
nmcli device wifi connect 'YOUR_WIFI_NAME' password 'YOUR_PASSWORD'
ping -c 3 google.com
```

STEP4 Remove GNOME

Strip the pre-installed GUI

```
sudo apt remove -y gnome* gdm3 ubuntu-desktop
sudo apt autoremove -y && sudo apt autoclean
sudo systemctl disable gdm3 2>/dev/null || true
sudo systemctl set-default multi-user.target
sudo reboot
```

STEP5 Set hostname

Brand the machine

```
sudo hostnamectl set-hostname tamberbox
echo '127.0.1.1 tamberbox' | sudo tee -a /etc/hosts
```

STEP6 Enable SSH from Windows

For copy-paste commands

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
ip a | grep inet   # note your IP e.g. 192.168.1.xxx

# On Windows PowerShell:
ssh praknoor@192.168.1.xxx
```

> ✓ *Always use SSH from Windows for the rest of this guide — you can copy-paste commands directly.*

Phase 2 — GPU Drivers

> ⚠ *GPU firmware is critical — without it Hyprland will not start at all.*

What we discovered about the RK3588 GPU

The RK3588 uses the Mali-G610 GPU with the Panthor DRM driver (built into the vendor kernel). It does NOT use /dev/mali0 — instead it exposes /dev/dri/card0, card1, card2 and renderD128/129/130. This confused us initially.

STEP7 **Install Mali firmware**
The package name differs from what the guide originally said

✗ ERROR: mali-g610-firmware package not found

```
Error: E: Unable to locate package mali-g610-firmware
```

Fix: Add the rockchip-multimedia PPA first, then download firmware manually

```
# Add working PPA
sudo add-apt-repository ppa:liujianfeng1994/rockchip-multimedia
sudo apt update
sudo apt install -y rockchip-multimedia-config

# Download firmware directly (package not in PPA for Noble)
cd /lib/firmware
sudo wget https://github.com/JeffyCN/mirrors/raw/libmali/firmware/g610/mali_csffw.bin
sudo mv mali_csffw.bin.1 mali_csffw.bin 2>/dev/null || true

# Verify GPU is detected
ls /dev/dri/
dmesg | grep -i mali
# Should show: panthor fbeeeeee.gpu-panthor: [drm] mali-g610 id 0xa867
```

> ✓ *card0/1/2 and renderD128/129/130 existing = GPU working. /dev/mali0 does NOT exist on this kernel — that is normal.*

Phase 3 — Hyprland (Compiled from Source)

> ⚠ *There is NO pre-built Hyprland binary for Ubuntu Noble ARM64. No PPA works. Compilation from source is the only option. This took approximately 3 hours total.*

Install dependencies

```
yay -S hyprland-git  # DO NOT USE on Armbian
# ^ This is for Arch only. On Armbian:

sudo apt install -y cmake meson ninja-build pkg-config git build-essential \
  libwayland-dev wayland-protocols libxkbcommon-dev libpixman-1-dev \
  libinput-dev libudev-dev libseat-dev libdrm-dev libgbm-dev \
  libvulkan-dev glslang-tools libxcb-util-dev libxcb-ewmh-dev \
  libxcb-icccm4-dev libx11-xcb-dev libxcb-xinput-dev libxcb-present-dev \
  libxcb-dri3-dev libxcb-res0-dev libtomlplusplus-dev libcairo2-dev \
  libpango1.0-dev libliftoff-dev libdisplay-info-dev hwdata \
  gcc-14 g++-14 libpugixml-dev libzip-dev librsvg2-dev libre2-dev
```

Errors encountered during Hyprland compilation

✗ ERROR: cmake too old

```
Error: CMake 3.28 — requires 3.30+
```

Fix: Download official ARM64 binary: wget cmake-3.30.5-linux-aarch64.sh from Kitware releases

```
wget
https://github.com/Kitware/CMake/releases/download/v3.30.5/cmake-3.30.5-linux-aarch64.sh
sudo ./cmake-3.30.5-linux-aarch64.sh --prefix=/usr/local --skip-license
```

✗ ERROR: GCC too old

```
Error: <print> header missing — requires GCC 14+
```

Fix: sudo apt install -y gcc-14 g++-14 && sudo update-alternatives ...

✗ ERROR: hyprland-protocols / hypr* libs missing

```
Error: Package 'libhyprlang-dev' has no installation candidate
```

Fix: Build all hyprwm dependencies from source in this order

Build order (dependency chain — must follow exactly):

```
# 1. hyprwayland-scanner
git clone https://github.com/hyprwm/hyprwayland-scanner.git
cd hyprwayland-scanner && cmake -B build && cmake --build build -j$(nproc) && sudo cmake
--install build
```

```
# 2. hyprutils
cd ~ && git clone https://github.com/hyprwm/hyprutils.git
cd hyprutils && cmake -B build && cmake --build build -j$(nproc) && sudo cmake --install
build

# 3. hyprlang
cd ~ && git clone https://github.com/hyprwm/hyprlang.git
cd hyprlang && cmake -B build && cmake --build build -j$(nproc) && sudo cmake --install
build

# 4. hyprcursor
cd ~ && git clone https://github.com/hyprwm/hyprcursor.git
cd hyprcursor && cmake -B build && cmake --build build -j$(nproc) && sudo cmake --install
build

# 5. hyprgraphics
cd ~ && git clone https://github.com/hyprwm/hyprgraphics.git
cd hyprgraphics && cmake -B build && cmake --build build -j$(nproc) && sudo cmake
--install build

# 6. aquamarine (needs libinput 1.28+ — see below)
cd ~ && git clone https://github.com/hyprwm/aquamarine.git
cd aquamarine && cmake -B build && cmake --build build -j$(nproc) && sudo cmake --install
build
```

✗ ERROR: libinput too old

```
Error: libinput 1.25.0 — aquamarine requires 1.26.0+, Hyprland needs 1.28.0+
```

Fix: Build libinput 1.28.0 from source — took about 5 minutes

```
sudo apt install -y libmtdev-dev libevdev-dev libgudev-1.0-dev python3-jinja2
cd ~
wget
https://gitlab.freedesktop.org/libinput/libinput/-/archive/1.28.0/libinput-1.28.0.tar.gz
tar xf libinput-1.28.0.tar.gz && cd libinput-1.28.0
meson setup build --prefix=/usr/local -Ddocumentation=false -Dtests=false
-Ddebug-gui=false
ninja -C build && sudo ninja -C build install
sudo ldconfig

# Force linker to find new version
sudo rm /lib/aarch64-linux-gnu/libinput.so
sudo rm /lib/aarch64-linux-gnu/libinput.so.10
sudo ldconfig
```

✗ ERROR: wayland-server too old

```
Error: wayland-server 1.22.0 — requires 1.22.90+
```

Fix: Build wayland 1.23.1 from source

```
sudo apt install -y libexpat1-dev libffi-dev
cd ~ && git clone https://gitlab.freedesktop.org/wayland/wayland.git
cd wayland && git checkout 1.23.1
meson setup build --prefix=/usr/local -Ddocumentation=false -Dtests=false
ninja -C build && sudo ninja -C build install && sudo ldconfig

# Force linker
sudo rm /lib/aarch64-linux-gnu/libwayland-server.so
```

```
sudo rm /lib/aarch64-linux-gnu/libwayland-server.so.0
echo '/usr/local/lib/aarch64-linux-gnu' | sudo tee /etc/ld.so.conf.d/local-aarch64.conf
sudo ldconfig
```

Final Hyprland build

```
sudo ldconfig
cd ~/Hyprland
git checkout v0.47.0
cmake --no-warn-unused-cli \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=/usr/local \
  -DNO_XWAYLAND=1 \
  -DCMAKE_EXE_LINKER_FLAGS='-L/usr/local/lib/aarch64-linux-gnu -L/usr/local/lib' \
  -S . -B ./build
cmake --build ./build -j$(nproc)    # takes 15-20 minutes
sudo cmake --install ./build

# Verify
Hyprland --version
# Should show: Hyprland 0.47.0
```

> ⚠ *Use v0.47.0 not main branch — latest Hyprland requires even newer deps not available on Noble*
>
> ⚠ *-DNO_XWAYLAND=1 is required — libxcb-errors not buildable on Noble. XWayland can be added later*

Phase 4 — Dotfiles & Desktop Setup

Install all dotfile dependencies

```
sudo apt install -y waybar cava kitty tofi thunar btop fastfetch cmus mpv
sudo apt install -y network-manager-gnome blueman pavucontrol
sudo apt install -y pipewire pipewire-pulse wireplumber
sudo apt install -y wl-clipboard grim slurp dunst
sudo apt install -y xdg-desktop-portal-hyprland xdg-desktop-portal-gtk
sudo apt install -y qt5-wayland qt6-wayland polkit-gnome
sudo apt install -y zsh swww
pip3 install pywal --break-system-packages

# Packages NOT in Ubuntu Noble repos — need alternatives:
# tofi: build from source (meson setup build && ninja -C build && sudo ninja -C build
install)
# fastfetch: add PPA ppa:zhangsongcui3371/fastfetch
# swww: download binary from GitHub releases (aarch64)
# nwg-look: build from source with golang
```

Apply dhaneshdutta dotfiles

```
git clone https://github.com/dhaneshdutta/dotfiles.git ~/dotfiles
cp -r ~/dotfiles/.config/* ~/.config/
cp ~/dotfiles/.zshrc ~/
cp ~/dotfiles/.bashrc ~/
```

Critical ARM fixes for hyprland.conf

> ⚠ *The dotfiles were made on x86 — these fixes are REQUIRED on OPi5 Ultra*

```
# ~/.config/hypr/hyprland.conf

# Monitor — auto detect
monitor=,preferred,auto,1

# ARM-specific env vars (add to env section)
env = WLR_RENDERER,vulkan
env = WLR_NO_HARDWARE_CURSORS,1
env = XDG_SESSION_TYPE,wayland
env = XDG_SESSION_DESKTOP,Hyprland
env = QT_QPA_PLATFORM,wayland
env = GDK_BACKEND,wayland
env = MOZ_ENABLE_WAYLAND,1

# Remove/comment these — not installed:
# exec-once = hypridle
# exec-once = pypr
# exec-once = /home/dhanesh/...  (wrong username!)

# Fix dhanesh paths to praknoor
sed -i 's|/home/dhanesh|/home/praknoor|g' ~/.config/waybar/style.css
sed -i 's|/home/dhanesh|/home/praknoor|g' ~/.config/waybar/config.jsonc
```

```
sed -i 's|/home/dhanesh|/home/praknoor|g' ~/.config/hypr/hyprland.conf
```

Fix waybar pywal CSS error

✗ ERROR: Waybar CSS import fails

```
Error: Failed to import /home/dhanesh/.cache/wal/colors-waybar.css: No such file
```

Fix: Create the CSS file with Tamberbox cyan colors

```
mkdir -p ~/.cache/wal
cat > ~/.cache/wal/colors-waybar.css << 'EOF'
@define-color color0 #1a1a2e;
@define-color color1 #00f5ea;
@define-color background #0a0a0f;
@define-color foreground #00f5ea;
EOF
```

Launch Hyprland

> ⚠ *Always launch from physical TTY keyboard — NOT from SSH. SSH has no Wayland socket.*

```
# On physical keyboard: Ctrl+Alt+F2 to switch to TTY2
# Log in, then:
Hyprland

# Inside Hyprland terminal (SUPER+Enter):
swww-daemon &
sleep 2 && swww img ~/.config/hypr/wallpapers/yourwallpaper.jpg
wal -i ~/.config/hypr/wallpapers/yourwallpaper.jpg -q
killall waybar && waybar &
```

Phase 5 — SDDM Login Manager + Dog Samurai Theme

Install SDDM

```
sudo apt install -y sddm qml-module-qtquick2 qml-module-qtquick-controls2
sudo systemctl enable sddm
sudo systemctl set-default graphical.target
```

Dog Samurai theme installation

```
git clone https://github.com/Darkkal44/qylock.git ~/qylock

# Use the Qt5 pre-converted version (not the default)
sudo cp -r ~/qylock/themes-qt5/dog-samurai /usr/share/sddm/themes/

# Install missing QML modules
sudo apt install -y qml-module-qt-labs-folderlistmodel \
  qml-module-qtquick-controls qml-module-qtquick-layouts \
  qml-module-qtquick-window2

sudo tee /etc/sddm.conf.d/theme.conf << 'EOF'
[Theme]
Current=dog-samurai
EOF
```

Fix animated video background — SOLVED

The Dog Samurai theme has a video background (bg.mp4) that did not play on ARM. This was a documented first fix for SDDM video on ARM64 Ubuntu Noble + RK3588.

✗ ERROR: SDDM video not playing

```
Error: Qt5 Multimedia had no GStreamer backend —
/usr/lib/.../qt5/plugins/mediaservice/ was EMPTY
```

Fix: Install libqt5multimedia5-plugins — this was the missing package

```
sudo apt install -y libqt5multimedia5-plugins

# Verify the mediaservice plugins now exist
ls /usr/lib/aarch64-linux-gnu/qt5/plugins/mediaservice/
# Should show: libgstmediaplayer.so libgstaudiodecoder.so etc.

sudo systemctl restart sddm
```

> ✓ *This fix (libqt5multimedia5-plugins) is believed to be the first documented solution for SDDM video on ARM64 Ubuntu Noble with RK3588.*

Phase 6 — Plymouth Boot Screen

> ⚠ *Plymouth on Armbian RK3588 vendor kernel is tricky. Follow these steps EXACTLY.*

The boot parameter problem

Armbian on RK3588 does NOT use GRUB. It uses armbianEnv.txt. The original guide said 'update-grub' which does nothing. The correct approach:

✗ ERROR: update-grub does nothing

```
Error: /etc/default/grub does not exist on Armbian RK3588
Fix:   Edit /boot/armbianEnv.txt instead
```

```
# View current boot config
cat /boot/armbianEnv.txt

# Add quiet splash
sudo sed -i 's/extraargs=cma=256M/extraargs=cma=256M quiet splash/' /boot/armbianEnv.txt

# Disable Armbian bootlogo (conflicts with Plymouth)
sudo sed -i 's/bootlogo=true/bootlogo=false/' /boot/armbianEnv.txt

# Change console to avoid kernel log fighting Plymouth
sudo sed -i 's/verbosity=1/verbosity=0/' /boot/armbianEnv.txt
sudo sed -i 's/console=both/console=serial/' /boot/armbianEnv.txt
```

Plymouth was working then stopped — cause found

Plymouth stopped working after we modified armbianEnv.txt multiple times. The fix was reverting to console=serial (removing console=both) which was causing the HDMI kernel log to fight Plymouth for the display.

Install and configure Plymouth

```
sudo apt install -y plymouth plymouth-themes fonts-dejavu

# Clone themes
git clone https://github.com/adi1090x/plymouth-themes.git ~/plymouth-themes

# Copy Cross HUD theme and create Tamberbox fork
sudo cp -r ~/plymouth-themes/pack_1/cross_hud /usr/share/plymouth/themes/
sudo cp -r /usr/share/plymouth/themes/cross_hud /usr/share/plymouth/themes/tamberbox
sudo mv /usr/share/plymouth/themes/tamberbox/cross_hud.plymouth \
        /usr/share/plymouth/themes/tamberbox/tamberbox.plymouth
```

```
# Write theme descriptor
sudo tee /usr/share/plymouth/themes/tamberbox/tamberbox.plymouth << 'EOF'
```

```
[Plymouth Theme]
Name=Tamberbox
Description=Tamberbox AI Laptop Boot Screen
ModuleName=script

[script]
ImageDir=/usr/share/plymouth/themes/tamberbox
ScriptFile=/usr/share/plymouth/themes/tamberbox/cross_hud.script
EOF
```

```
# Prepend Tamberbox branding to animation script
sudo tee /tmp/brand.txt << 'EOF'
title_font = "DejaVu Sans Bold 28";
sub_font   = "DejaVu Sans 14";
title_image = Image.Text("TAMBERBOX", 1, 1, 1, 1, title_font);
title_sprite = Sprite();
title_sprite.SetImage(title_image);
title_sprite.SetX(Window.GetWidth() / 2 - title_image.GetWidth() / 2);
title_sprite.SetY(Window.GetHeight() / 2 + 120);
sub_image = Image.Text("AI Laptop | Built by Praknoor", 0.6, 0.6, 0.6, 1, sub_font);
sub_sprite = Sprite();
sub_sprite.SetImage(sub_image);
sub_sprite.SetX(Window.GetWidth() / 2 - sub_image.GetWidth() / 2);
sub_sprite.SetY(Window.GetHeight() / 2 + 158);
EOF

sudo bash -c 'cat /tmp/brand.txt /usr/share/plymouth/themes/tamberbox/cross_hud.script >
/tmp/m.script && cp /tmp/m.script /usr/share/plymouth/themes/tamberbox/cross_hud.script'
```

```
# Register theme
sudo update-alternatives --install /usr/share/plymouth/themes/default.plymouth \
  default.plymouth /usr/share/plymouth/themes/tamberbox/tamberbox.plymouth 100
sudo update-alternatives --config default.plymouth   # select tamberbox

# Set in plymouthd.conf
sudo mkdir -p /etc/plymouth
sudo tee /etc/plymouth/plymouthd.conf << 'EOF'
[Daemon]
Theme=tamberbox
ShowDelay=0
DeviceTimeout=8
EOF

# Rebuild initramfs
sudo update-initramfs -u -k all
sudo reboot
```

Phase 7 — AI Features

Ollama — Local LLM

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl enable --now ollama

# Pull models (pick based on speed vs quality)
ollama pull qwen2.5:1.5b   # 1.9GB — fastest, recommended
ollama pull qwen2.5:3b     # 2.0GB — better quality
ollama pull mistral        # 4.1GB — most capable

# Create Tamberbox personality
mkdir -p ~/.local/share/ollama
cat > ~/.local/share/ollama/tamberbox.modelfile << 'EOF'
FROM qwen2.5:1.5b
SYSTEM You are Tamberbox AI, built by Praknoor on an Orange Pi 5 Ultra ARM laptop.
Be brief and direct. You run on 8GB RAM with a 6 TOPS NPU.
EOF
ollama create tamberbox -f ~/.local/share/ollama/tamberbox.modelfile

# Test
ollama run tamberbox 'Introduce yourself'
```

RKNN Toolkit2 — NPU acceleration

```
# Clone the toolkit
git clone https://github.com/airockchip/rknn-toolkit2.git ~/rknn-toolkit2

# Install the correct wheel for Python 3.12 on aarch64
pip3 install
~/rknn-toolkit2/rknn-toolkit-lite2/packages/rknn_toolkit_lite2-2.3.2-cp312-cp312-manylinu
x_2_17_aarch64.manylinux2014_aarch64.whl --break-system-packages

# Verify NPU
python3 -c "from rknnlite.api import RKNNLite; r=RKNNLite(); print('NPU OK')"
dmesg | grep rknpu   # should show: Initialized rknpu 0.9.8
```

> ⚠ *The rknn-toolkit2 folder structure changed — the packages are in rknn-toolkit-lite2/packages/ not rknn_toolkit_lite2/packages/*

Whisper.cpp — offline speech to text

```
sudo apt install -y libsdl2-dev
git clone https://github.com/ggerganov/whisper.cpp.git ~/whisper.cpp
cd ~/whisper.cpp
make -j$(nproc)   # uses ARM NEON — takes ~5 minutes

# Download small English model
bash ./models/download-ggml-model.sh small.en

# Install
sudo cp build/bin/whisper-cli /usr/local/bin/whisper-cpp
```

```
mkdir -p ~/.local/share/whisper
cp models/ggml-small.en.bin ~/.local/share/whisper/

# Test
arecord -d 5 -f cd /tmp/test.wav
whisper-cpp /tmp/test.wav --model ~/.local/share/whisper/ggml-small.en.bin
```

Tamberbox AI script — voice assistant

```
sudo apt install -y espeak espeak-ng alsa-utils
pip3 install SpeechRecognition pyaudio requests --break-system-packages

# The tamberbox-ai.py script ties everything together:
# Voice input (Whisper) → AI brain (Ollama) → Voice output (espeak)
python3 ~/tamberbox-ai.py 'what is the orange pi 5 ultra'
```

Tamberbox AI GUI — Web interface

```
pip3 install flask --break-system-packages

# Launch the web GUI
cd ~/TamberboxAI && python3 app.py &

# Open in Chromium inside Hyprland
chromium --no-sandbox --app=http://localhost:7777

# Or add to Hyprland autostart:
exec-once = sleep 5 && ~/TamberboxAI/start.sh
```

Phase 8 — Tamberbox Branding & Customization

Sci-fi MOTD terminal banner

```
# /etc/update-motd.d/10-armbian-header
# Disable old Armbian banners:
sudo chmod -x /etc/update-motd.d/30-armbian-sysinfo 2>/dev/null || true
sudo chmod -x /etc/update-motd.d/35-armbian-tips 2>/dev/null || true
sudo chmod -x /etc/update-motd.d/40-armbian-updates 2>/dev/null || true

# Features: TAMBERBOX ASCII art, boot sequence style log,
# CPU/RAM/NPU/GPU/OS/AI info, 'The machine obeys. The mind commands.'

run-parts /etc/update-motd.d/   # test it
```

Live wallpaper with mpvpaper

```
# Build mpvpaper (uses meson, not cmake)
sudo apt install -y libmpv-dev meson ninja-build
cd /tmp && git clone https://github.com/GhostNaN/mpvpaper.git
cd mpvpaper && meson setup build && ninja -C build && sudo ninja -C build install

# Download wallpaper video (e.g. Zenitsu from yt-dlp)
sudo apt install -y yt-dlp
mkdir -p ~/wallpapers
yt-dlp 'https://moewalls.com/...' -o ~/wallpapers/wallpaper.mp4

# Add to hyprland.conf autostart
exec-once = mpvpaper -o 'no-audio loop' '*' ~/wallpapers/wallpaper.mp4
```

Cursor themes

```
# Install win2xcur converter
pip3 install win2xcur --break-system-packages

# Download cursor pack (ctrlcat0xx/cursors on GitHub)
git clone https://github.com/ctrlcat0xx/cursors.git ~/cursors

# Unzip and convert
unzip ~/cursors/cursor_pack_1/cinnamon_cursors.zip -d /tmp/cinnamon
mkdir -p ~/.icons/Cinnamon/cursors
win2xcur /tmp/cinnamon/**/*.cur /tmp/cinnamon/**/*.ani -o ~/.icons/Cinnamon/cursors/

# Apply in Hyprland
env = XCURSOR_THEME,Cinnamon
env = XCURSOR_SIZE,32
```

> ⚠ *Sweezy cursor .ani files are NOT standard Windows ANI format — win2xcur fails on them. Use ctrlcat0xx/cursors instead.*

Quick Reference — Commands You'll Use Daily

Hyprland keybinds

Keybind	Action
`SUPER + Enter`	Open Kitty terminal
`SUPER + Space`	Tofi app launcher
`SUPER + Q`	Close window
`SUPER + 1-9`	Switch workspace
`SUPER + Shift + 1-9`	Move window to workspace
`SUPER + Shift + S`	Screenshot
`SUPER + F`	Fullscreen
`SUPER + V`	Toggle floating
`SUPER + E`	Thunar file manager
`SUPER + M`	Exit Hyprland

Useful diagnostic commands

```
# Check Hyprland logs
cat ~/.cache/hyprland/hyprland.log | tail -50

# GPU status
ls /dev/dri/          # card0/1/2 and renderD128/129/130
dmesg | grep -i mali  # should show mali-g610

# NPU status
dmesg | grep rknpu    # should show: Initialized rknpu 0.9.8
python3 -c "from rknnlite.api import RKNNLite; r=RKNNLite(); print('NPU OK')"

# Ollama
ollama list
ollama run tamberbox 'hi'
systemctl status ollama

# System info
fastfetch
btop
```

Boot config file — critical

```
cat /boot/armbianEnv.txt

# What it should look like for Plymouth to work:
verbosity=0
bootlogo=false
```

```
console=serial
extraargs=cma=256M quiet splash
overlay_prefix=rockchip-rk3588
overlays=panthor-gpu
fdtfile=rockchip/rk3588-orangepi-5-ultra.dtb
```

Known Issues & Limitations

Known limitations on Armbian Noble + RK3588

- XWayland: disabled (libxcb-errors not buildable on Noble) — X11 apps won't run
- Hyprland must be compiled from source every update — no ARM64 package exists
- mpvpaper wallpaper only works inside Hyprland session (not from SSH)
- Sweezy cursor .ani files are proprietary and don't convert to Linux XCursor format
- NVMe migration: still on SD card — recommended to migrate for better performance
- Plymouth: works but requires verbosity=0 and console=serial in armbianEnv.txt

What's still to do

- NVMe migration — move OS from SD card to NVMe SSD for 10x faster storage
- PS4 UI gaming mode — d4v1-sudo/Ps4-UI with PPSSPP game tiles (in progress)
- OpenJarvis full desktop voice control — xdotool + AT-SPI2 wiring
- NPU-accelerated inference — convert models to RKNN format for 6 TOPS speed
- Hyprlock screen locker — configure with Tamberbox branding